Project - WireGuard VPN user authentication

André Zúquete

2026-02-13

1 Introduction

This project focuses on designing and implementing a secure user authentication protocol for WireGuard VPN.

2 WireGuard VPN

WireGuard implements a secure tunneling mechanism on a peer-to-peer fashion. Any of the peers can initiate such tunnel. Peers authenticate themselves with uncertified asymmetric key pairs, which are used to (frequently) setup the tunnel session key with a fast protocol called NoiseIK, which is a variant of the multipurpose two-message Noise protocol.

When WireGuard VPNs are used by users (and not between networks or servers), to deploy a host-to-host VPN (e.g. when a user accesses a server through a VPN) or host-to-net VPN (e.g. when a user accesses a network of hosts through a VPN), there is not really a peer-to-peer configuration. Instead, there is a client-server configuration, because the initiator is always the user host and the responder is the server host and the network gateway.

Thus, the user needs to configure their WireGuard interface with 6 mandatory parameters:

- About itself:

1. Its secret key;
2. The (private) IP address of its tunnel endpoint.

- About the peer (server/gateway):

3. The public key;
4. The public IP address;
5. The UDP port used by the WireGuard tunnel setup protocol;
6. A pre-shared, symmetric key.

On the contrary, the server/gateway needs to configure its WireGuard interface with different, but also 6 mandatory parameters:

- About itself:

1. Its secret key;
2. The (private) IP address of its tunnel endpoint.
3. The UDP port used by the WireGuard tunnel setup protocol;

- About the peer (user host):

4. The public key;
5. A (private) IP address, or set of addresses (given a network mask);
6. A pre-shared, symmetric key.

As we can see, the server side configuration does not scale for a large and very dynamic population of users. And even of the user side, it is complex to ensure that there are no collisions among their private IP addresses.

3 Project Objectives

The goal of the project is to create a secure user authentication protocol upon which new WireGuard interface configurations will be created and deployed. For that, there will be a UserAuthn application (for the users hosts) and a SrvAuthn application (for the server/gateway hosts).

The UserAuthn application will have 2 parameters:

- WireGuard interface;
- A username;
- base configuration about the server/gateway.

The base configuration will have a partial definition of a WireGuard peer (the server/gateway): public key (3), public IP address (4); and UDP port (5). These are to be used to build the 4-parameter WireGuard interface peer definition. The other 3 parameters required to configure a WireGuard client (1, 2 and 6) must be either generated locally on each execution (6) or provided by the SrvAuthn application (1 and 2).

The SrvAuthn application will have 2 parameters:

- WireGuard interface;
- base configuration file about itself (parameters 1, 2 and 3).

All user host's configurations will be built upon a proper user authentication (parameters 4, 5 and 6 per host).

You can used a fixed transport port for implementing the dialog between the UserAuthn and SrvAuthn application. Or you can provide its configuration with an extra, optional parameter in both applications.

The user authentication protocol must leverage MFA (Multi-Factor Authentication) and RBA (Risk-Based Authentication). For MFA you must deploy 3 mandatory technologies: password-based (challenge-response -based), TOTP and FIDO2. Consider the first the less robust (but probably more convenient) and the last the most robust (by also the less robust to loss, theft or malfunction).

For the user authentication you will have to use an IdP, either through SAML or OAuth 2.0. This IdP is the component that implements MFA and RBA, and the SrvAuthn application trusts on its behavior.

For RBA consider using the following key indicators for taking a more demanding authentication: IP address of the user host (same/different, same/different Autonomous Service), authentication failures, ordinary time of the day for VPN creation, etc.

4 Implementation details

The user authentication is easier to implement with a browser, because it simplifies the MFA deployment.

The project must use the clients' knowledge of the servers/gateways's WireGuard public key as a source of trust.

For the IdP authentication, select the best approach. One interesting possibility is to explore HTTP interactions through a secure tunnel (e.g. via TLS) created between the UserAuthn and the SrvAuthn application. In this case, the user browser can use HTTP (instead of HTTPS), connects to a TCP port deployed by UserAuthn (client tunnel endpoint), and the SrvAuthn redirects the traffic from the tunnel to the IdP. This eliminates the user side need to authenticate the IdP; all that is necessary is to authenticate the SrvAuthn application with the WireGuard's public key (something similar is provided by SSH tunnels).

For exploring OAuth 2.0, it may be interesting to select the Device Authorization Grant flow, since it simplifies the integration of the UserAuthn application with a browser.

The IdP MFA and RBA orchestration must be done with PAM. You must implement a PAM module, to be launched from the IdP PAM profile configuration, that provides instructions to the IdP about the authentication methods to be used and their sequence. You can use the PAM callback function to give instructions to the IdP and for collecting authentication results (success / failure / no availability).

As in any IdP, an enrolment process must exist, as well as a credential update possibility. Provide an interface for it, but without any security-related validations. Assume that that interface is internal and exclusive for an helpdesk service.

5 Project execution, delivery and grading

This work is expected to be implemented by a group of 3 students and MUST be delivered using eLarning. Delivery should consist of a ZIP file with at least 4 folders and a file:

- client: contains the code that is used exclusively by VPN clients;
- server: contains the code that is used exclusively in VPN servers;
- idp: contains the code that is used exclusively in the IdP;
- common: contains the code that handles common support features;
- README.md: contains a succinct report of the project design and implementation, which includes a justified description of all implemented features. It must clearly identify the authors (number and name), their percentual contribution to the overall project work and the features not implemented at all or partially implemented.

Projects will be graded according to:

- The suitability/correctness of the options taken (70%);
- The documentation produced (30%).

A maximum of 10% bonus will be granted to projects that implement some interesting, non-mandatory features.

This project is expected to be authored by the students enrolled in the course. The use of existing code snippets, applications, or any other external functional element without proper acknowledgement is strictly forbidden. If any content lacking proper acknowledgment is found in other sources, the current rules regarding plagiarism will be followed.

6 References

- WireGuard Quick Start
- The Linux-PAM Module Writers' Guide